

Artificial Intelligence

Lecture 6, Chapter 5 Adversarial Search

Supta Richard Philip

<https://suptaphilip.github.io/>

Department of CS
AIUB

AIUB, June 2026



Table of Contents

- 1 Games
- 2 Optimal Decisions in Games
- 3 Alpha-Beta Pruning
- 4 Imperfect Real-Time Decisions
- 5 Stochastic Games
- 6 References



Table of Contents

- 1 Games
- 2 Optimal Decisions in Games
- 3 Alpha-Beta Pruning
- 4 Imperfect Real-Time Decisions
- 5 Stochastic Games
- 6 References



Games

- Multiagent environments, in which each agent needs to consider the actions of other agents and how they affect its own welfare.
- In Competitive environments, agents goals are in conflict, giving rise to adversarial search problems—often known as games.
- Mathematical game theory, a branch of economics, views any multiagent environment as a game.
- In AI, the most common games are deterministic, turn taking, two-player, zero-sum games of perfect information (such as chess).
- In our terminology, Games means deterministic, fully observable environments in which two agents act alternately and in which the utility values at the end of the game are always equal and opposite. For example, if one player wins a game of chess, the other player necessarily loses.



Games

- For example, chess has an average branching factor of about 35, and games often go to 50 moves by each player, so the search tree has about 35^{100} or 10^{154} nodes.
- Pruning allows us to ignore portions of the search tree that make no difference to the final choice, and heuristic evaluation functions allow us to approximate the true utility of a state without doing a complete search.



Components of a Game

A game can be formally defined as a kind of search problem with the following elements:

- S_0 : The initial state, which specifies how the game is set up at the start.
- $\text{PLAYER}(s)$: Defines which player has the move in a state.
- $\text{ACTIONS}(s)$: Returns the set of legal moves in a state.
- $\text{RESULT}(s, a)$: The transition model, which defines the result of a move.
- $\text{TERMINAL-TEST}(s)$: A terminal test, which is true when the game is over and false otherwise. States where the game has ended are called terminal states.
- $\text{UTILITY}(s, p)$: A utility function defines the final numeric value for a game that ends in terminal state s for a player p .



Components of a Game

- The initial state, ACTIONS function, and RESULT function define the game tree for the game.
- A Game tree where the nodes are game states and the edges are moves.
- Figure shows part of the game tree for tic-tac-toe (noughts and crosses).
- From the initial state, MAX has nine possible moves. Play alternates between MAX's placing an X and MIN's placing an O until we reach leaf nodes corresponding to terminal states such that one player has three in a row or all the squares are filled.
- The number on each leaf node indicates the utility value of the terminal state from the point of view of MAX;
- High values are assumed to be good for MAX and bad for MIN.



Game Tree of Tic-Tac-Toe

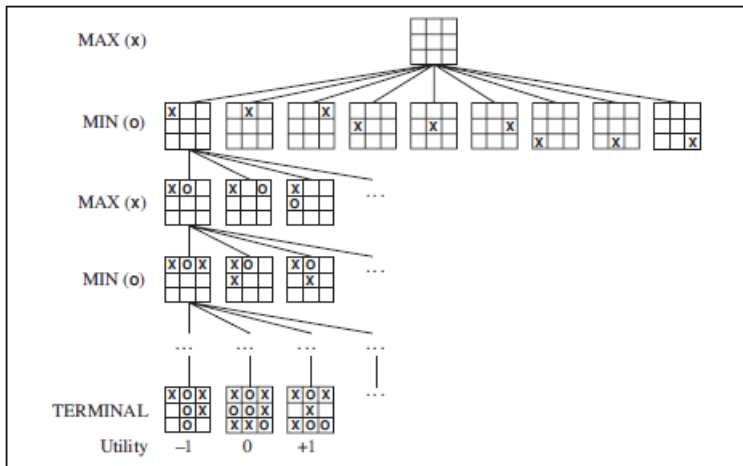


Figure 5.1 A (partial) game tree for the game of tic-tac-toe. The top node is the initial state, and MAX moves first, placing an X in an empty square. We show part of the tree, giving alternating moves by MIN (O) and MAX (X), until we eventually reach terminal states, which can be assigned utilities according to the rules of the game.



Table of Contents

- 1 Games
- 2 Optimal Decisions in Games
- 3 Alpha-Beta Pruning
- 4 Imperfect Real-Time Decisions
- 5 Stochastic Games
- 6 References

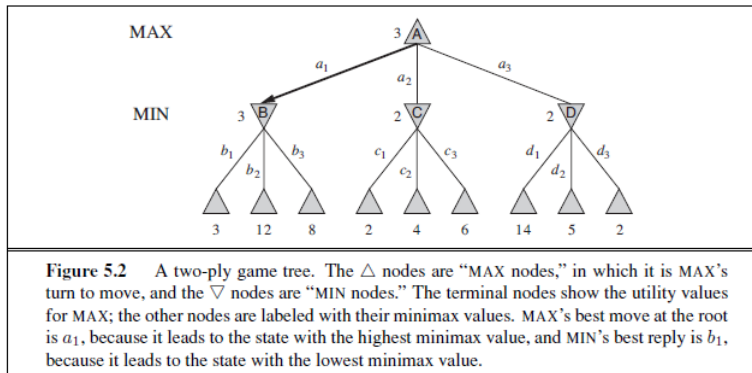


Optimal Decisions in Games

- In a normal search problem, the optimal solution would be a **sequence of actions leading to a goal state** whereas in Games the optimal solution would be a **sequence of actions leading to a terminal state** that is a win.
- In adversarial search, MIN has something to say about it.
- MAX therefore must find a **contingent strategy** which specifies MAX's move in the initial state, then MAX's moves in the states resulting from every possible response by MIN, then MAX's moves in the states resulting from every possible response by MIN to those moves, and so on.



Optimal Decisions in Games



- Given a game tree, the optimal strategy can be determined from the minimax value of each node, which we write as $\text{MINIMAX}(n)$.



Optimal Decisions in Games

- The minimax value of a node is the utility (for MAX) of being in the corresponding state, assuming that both players play optimally from there to the end of the game.
- Obviously, the minimax value of a terminal state is just its utility.
- Furthermore, given a choice, MAX prefers to move to a state of **maximum value**, whereas MIN prefers a state of **minimum value**.

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$



The minimax algorithm

```
function MINIMAX-DECISION(state) returns an action  
  return  $\arg \max_{a \in \text{ACTIONS}(s)} \text{MIN-VALUE}(\text{RESULT}(\text{state}, a))$ 
```

```
function MAX-VALUE(state) returns a utility value  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow -\infty$   
  for each a in ACTIONS(state) do  
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a)))$   
  return v
```

```
function MIN-VALUE(state) returns a utility value  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow \infty$   
  for each a in ACTIONS(state) do  
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a)))$   
  return v
```

Figure 5.3 An algorithm for calculating minimax decisions. It returns the action corresponding to the best possible move, that is, the move that leads to the outcome with the best utility, under the assumption that the opponent plays to minimize utility. The functions MAX-VALUE and MIN-VALUE go through the whole game tree, all the way to the leaves, to determine the backed-up value of a state. The notation $\arg \max_{a \in S} f(a)$ computes the element *a* of set *S* that has the maximum value of *f*(*a*).



Complexity of minimax algorithm

- The minimax algorithm computes the minimax decision from the current state. It uses a simple recursive computation of the minimax values of each successor state, directly implementing the defining equations.
- The recursion proceeds all the way down to the leaves of the tree, and then the minimax values are backed up through the tree as the recursion unwinds.
- The minimax algorithm performs a complete depth-first exploration of the game tree.
- If the maximum depth of the tree is m and there are b legal moves at each point, then the time complexity of the minimax algorithm is $O(b^m)$.
- The space complexity is $O(bm)$ for an algorithm that generates all actions at once, or $O(m)$ for an algorithm that generates actions one at a time.



minimax exercise

- Apply the minimax algorithm for finding the best action for the max player at the root.

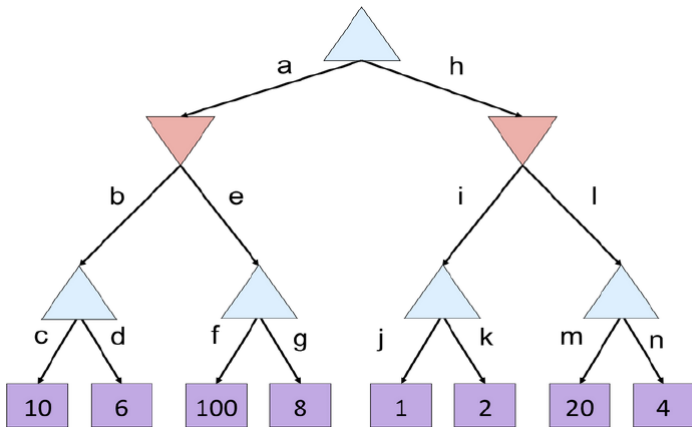


Table of Contents

- 1 Games
- 2 Optimal Decisions in Games
- 3 Alpha-Beta Pruning
- 4 Imperfect Real-Time Decisions
- 5 Stochastic Games
- 6 References



Alpha-Beta Pruning

- The problem with minimax search is that the number of game states it has to examine is exponential in the depth of the tree.
- Unfortunately, we can't eliminate the exponent, but it turns out we can effectively cut it in half.
- The trick is that it is possible to compute the correct minimax decision without looking at every node in the game tree.
- The particular technique we examine is called **alpha-beta pruning**.
- α = the value of the best (i.e., highest-value) choice we have found so far at any choice point along the path for MAX.
- β = the value of the best (i.e., lowest-value) choice we have found so far at any choice point along the path for MIN.



alpha-beta pruning algorithm

```
function ALPHA-BETA-SEARCH(state) returns an action  
   $v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$   
  return the action in ACTIONS(state) with value v
```

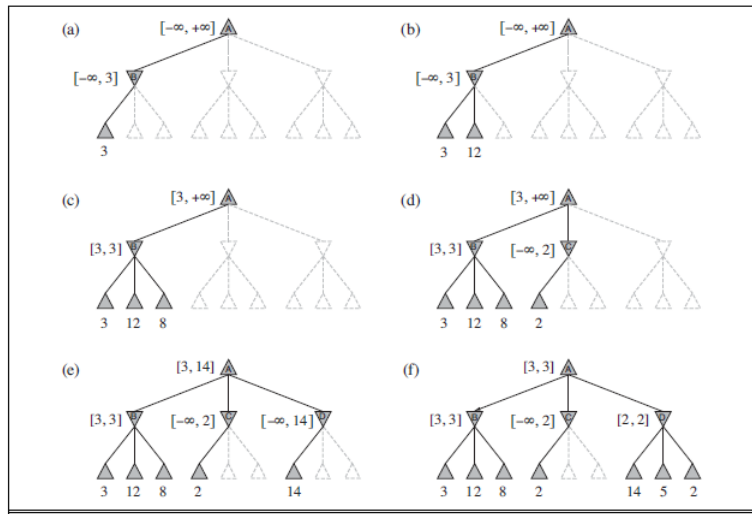
```
function MAX-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow -\infty$   
  for each a in ACTIONS(state) do  
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$   
    if  $v \geq \beta$  then return v  
     $\alpha \leftarrow \text{MAX}(\alpha, v)$   
  return v
```

```
function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow +\infty$   
  for each a in ACTIONS(state) do  
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$   
    if  $v \leq \alpha$  then return v  
     $\beta \leftarrow \text{MIN}(\beta, v)$   
  return v
```

Figure 5.7 The alpha-beta search algorithm. Notice that these routines are the same as the MINIMAX functions in Figure 5.3, except for the two lines in each of MIN-VALUE and MAX-VALUE that maintain α and β (and the bookkeeping to pass these parameters along).



alpha-beta pruning algorithm



alpha-beta exercise

- Consider the following tree representing a zero-sum game, where triangles pointing up are max nodes, triangles pointing down are min nodes, and squares are terminal nodes with the corresponding value of utility function for max player.
- Apply the minimax algorithm with alpha-beta pruning for finding the best action for the max player at the root.

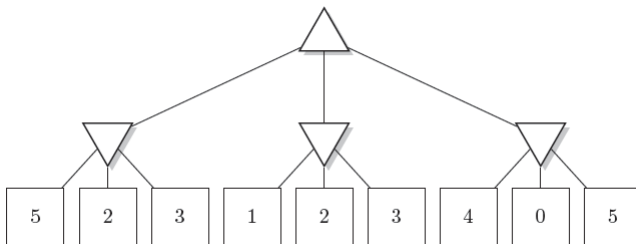


Table of Contents

- 1 Games
- 2 Optimal Decisions in Games
- 3 Alpha-Beta Pruning
- 4 Imperfect Real-Time Decisions**
- 5 Stochastic Games
- 6 References



Imperfect Real-Time Decisions

- The minimax algorithm generates the entire game search space, whereas the alpha-beta algorithm allows us to prune large parts of it.
- However, alpha-beta still has to search all the way to terminal states for at least a portion of the search space.
- This depth is usually not practical, because moves must be made in a reasonable amount of time—typically a few minutes at most.
- Claude Shannon's paper Programming a Computer for Playing Chess (1950) proposed instead that programs should cut off the search earlier and apply a heuristic evaluation function to states in the search, effectively turning nonterminal nodes into terminal leaves.



Imperfect Real-Time Decisions

In other words, the suggestion is to alter minimax or alpha-beta in two ways:

- replace the utility function by a heuristic evaluation function EVAL, which estimates the position's utility,
- replace the terminal test by a cutoff test that decides when to apply EVAL.
- That gives us the following for heuristic minimax for state s and maximum depth d :

$$\text{H-MINIMAX}(s, d) = \begin{cases} \text{EVAL}(s) & \text{if CUTOFF-TEST}(s, d) \\ \max_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if PLAYER}(s) = \text{MIN}. \end{cases}$$



Evaluation functions

- An evaluation function returns an estimate of the expected utility of the game from a given position, just as the heuristic functions.
- An inaccurate evaluation function will guide an agent toward positions that turn out to be lost.
- How exactly do we design good evaluation functions?
- First, the evaluation function should order the terminal states in the same way as the true utility function.
- Second, the computation must not take too long!
- Third, for nonterminal states, the evaluation function should be strongly correlated with the actual chances of winning.



Evaluation functions

- Most evaluation functions work by calculating various features of the state—for example, in chess, we would have features for the number of white pawns, black pawns, white queens, black queens, and so on.
- Mathematically, this kind of evaluation function is called a weighted linear function because it can be expressed as
$$EVAL(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s) =$$

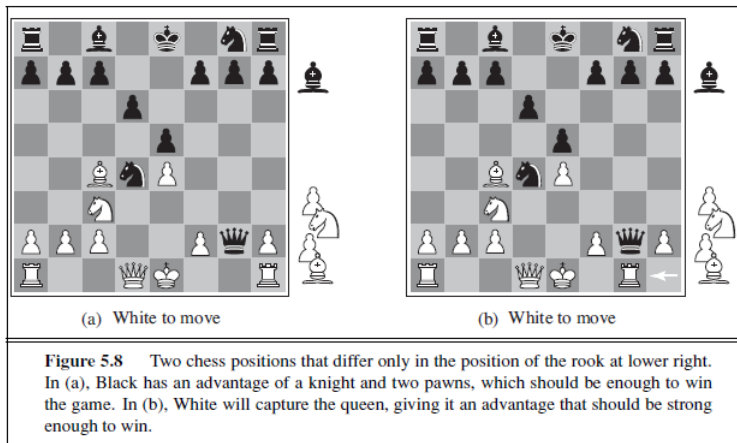
$$\sum_{i=1}^n w_i f_i(s)$$

where each w_i is a weight and each f_i is a feature of the position.



Evaluation functions

- For chess, the f_i could be the numbers of each kind of piece on the board, and the w_i could be the values of the pieces (1 for pawn, 3 for bishop, etc.).



Cutting off search

- The next step is to modify ALPHA-BETA-SEARCH so that it will call the heuristic EVAL function when it is appropriate to cut off the search.
- We replace the two lines that mention TERMINAL-TEST with the following line:
if CUTOFF-TEST(state, depth) then return EVAL(state)
- The most straightforward approach to controlling the amount of search is to set a fixed depth limit so that CUTOFF-TEST(state, depth) returns true for all depth greater than some fixed depth d .
- The depth d is chosen so that a move is selected within the allocated time.
- A more robust approach is to apply iterative deepening. When time runs out, the program returns the move selected by the deepest completed search.



Table of Contents

- 1 Games
- 2 Optimal Decisions in Games
- 3 Alpha-Beta Pruning
- 4 Imperfect Real-Time Decisions
- 5 Stochastic Games
- 6 References



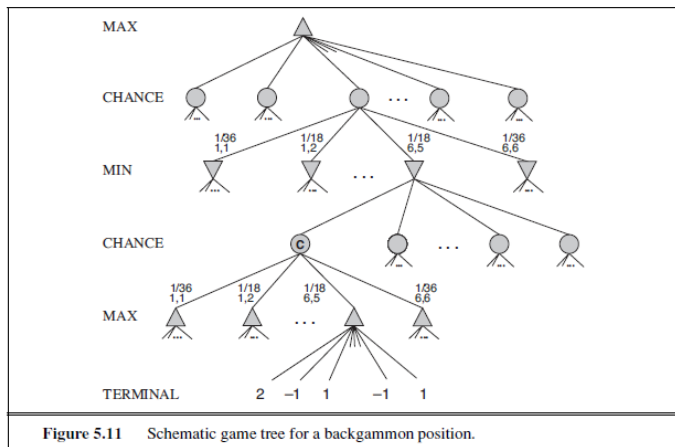
Stochastic Games

- In real life, many unpredictable external events can put us into unforeseen situations.
- Many games mirror this unpredictability by including a random element, such as the throwing of dice. We call these stochastic games.
- Backgammon is a typical game that combines luck and skill.
- A game tree in backgammon must include chance nodes in addition to MAX and MIN nodes. Chance nodes are shown as circles.



Stochastic Games

- we can not calculate minimax values of best position but only calculate the expected value of a position: the average over all possible outcomes of the chance nodes.



expectiminimax

- This leads us to generalize the minimax value for deterministic games to an expectiminimax value for games with chance nodes.

EXPECTIMINIMAX(s) =

$$\begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_a \text{EXPECTIMINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_a \text{EXPECTIMINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \\ \sum_r P(r) \text{EXPECTIMINIMAX}(\text{RESULT}(s, r)) & \text{if } \text{PLAYER}(s) = \text{CHANCE} \end{cases}$$

where r represents a possible dice roll (or other chance event) and $\text{RESULT}(s, r)$ is the same state as s , with the additional fact that the result of the dice roll is r .



expectiminimax Game Tree

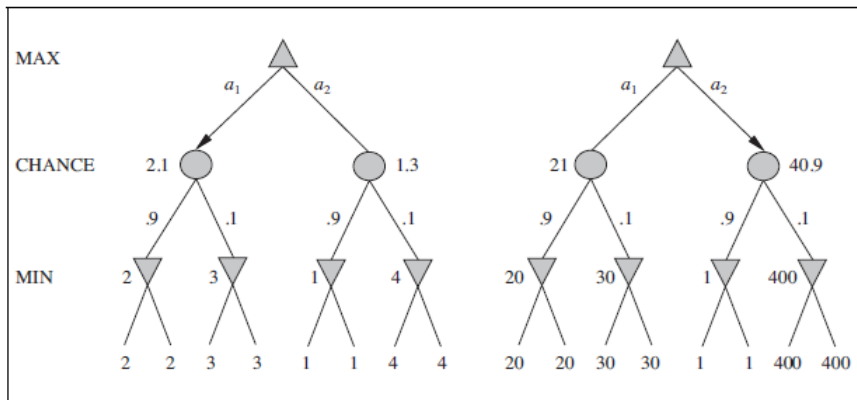


Figure 5.12 An order-preserving transformation on leaf values changes the best move.



Table of Contents

- 1 Games
- 2 Optimal Decisions in Games
- 3 Alpha–Beta Pruning
- 4 Imperfect Real-Time Decisions
- 5 Stochastic Games
- 6 References



References

-  Stuart Russell and Peter Norvig. 2021. Artificial Intelligence: A Modern Approach (4th ed.). Pearson Education Inc., Boston, MA.
-  Stuart Russell and Peter Norvig. 2009. Artificial Intelligence: A Modern Approach (3rd ed.). Prentice Hall.
-  Artificial Intelligence : Foundations of Computational, David L. Poole and Alan K. Mackworth, 3rd edition, Cambridge University Press, 2023.
-  Introduction to Artificial Intelligence, Nikhil Sharma, Josh Hug, Jacky Liang, and Henry Zhu.
-  <https://www.cs.cmu.edu/15281-s25>
-  <https://inst.eecs.berkeley.edu/cs188/su25/>

